

Team Project Ideas

Eight project briefs — one for every team

Mazen Barakat
INSTRUCTOR

How to use this document

Each team takes one idea. Every idea below is a complete brief: what the system is, who uses it, the **pages you must build**, **extra features** you can add for higher marks, and **AI ideas** that fit that system. Read your idea from top to bottom before you write a single line of code, then draw your database on paper.

All eight ideas are equal in difficulty. None of them is the "easy" one — you are marked on how well you build it, not on which one you picked.

Every project must have these — this is not a wish list

Requirement	What we expect to see
ASP.NET Core MVC	Controllers, Razor views, models, a shared layout, partial views and tag helpers. Not Razor Pages, and not a Web API with a separate front-end.
Database	EF Core Code-First with SQL Server, migrations committed to the repo, at least 6 related tables including a one-to-many and a many-to-many relation.
Seed data	The app must look alive the moment we open it: 20–30 realistic rows in every main table, with real images and real Arabic/English names. Empty screens lose marks.
Login & sign-up MANDATORY	Both are required in every project , with no exception: a sign-up page where a new user creates an account and a login page, built on ASP.NET Core Identity, plus sign-out and change-password. At least two roles , pages protected with <code>[Authorize]</code> , and a menu that changes depending on who signed in. The admin account is seeded , not created by signing up.
Full CRUD	Create, read, update and delete on at least 3 entities , with a confirmation step before every delete.
Search, filter, paging	Any list longer than ten rows needs a search box, at least one filter, sorting and paging.
Validation	Data annotations and <code>ModelState</code> on the server plus client-side validation. Messages appear beside the field, not in a wall of text at the top.
Error handling	Custom 404 and 500 pages that match your design. The yellow exception screen must never appear.
AI feature	A working assistant inside the app — pick one of the three levels on page 3 and one of the ideas listed under your project.

Read this table as a checklist and confirm it before you submit. Every row above is a requirement, not a suggestion. Sit together as a team, go through it line by line, and confirm that your project satisfies each one. Anything missing from this table is a missing requirement, and it is marked as missing however good the rest of the project is.

Optional — build these only if you have time — they are not required

- **File upload** — at least one real upload (a photo, a cover, a CV, a document) saved under `wwwroot` with a type and size check.
- **Dashboard** — one admin screen with real numbers pulled from the database and at least one chart (Chart.js is fine).

Split the work, not the project. The worst thing a team of three can do is agree that "one does the database, one does the back-end, one does the design" and then meet again on the last day. Split by **feature**: each member owns two or three complete screens end to end (model → controller → view), and you merge your branches every single day.

The design is a requirement, not a decoration

Confirm this before you write a single page. The way your application looks is graded exactly like the code behind it. We are not accepting a wireframe: the scaffolded page you get for free — a white background, a bare table and blue *Edit / Details / Delete* links — is a starting point, not a deliverable. When we open your project it must look like a real product that a real person would enjoy using: friendly, consistent and finished. A project that ignores this section is returned as incomplete, however well the back-end works.

The rules you are required to follow

- **Start from a real template.** Bootstrap 5 plus a free theme (AdminLTE, Tabler, SB Admin, or any free Bootstrap landing template). Do not design from zero — you have one week.
- **One layout for everything.** A single `_Layout.cshtml` with a proper navbar, a sidebar for the admin area, a footer and breadcrumbs, inherited by every page. Every screen must look like it belongs to the same application.
- **Pick a palette and stay inside it:** one main colour, one accent, and greys. Two Google fonts at most.
- **Cards, not naked tables.** Anything the visitor browses — products, doctors, rooms, books, jobs — belongs in an image card grid. Keep tables for the admin screens, and style them.
- **Real images and real data everywhere,** plus an icon set for the actions, and a fallback on every image so a missing file never shows a broken icon. `test test`, `Product 1 / Product 2` and "lorem ipsum" are not acceptable content.
- **Talk to the user:** a green toast after saving, a red alert after a failure, an "are you sure?" confirmation before deleting, and a friendly **empty state** ("No orders yet — start shopping") instead of a blank table.
- **Be responsive.** Open every page at phone width: nothing may overflow sideways, and nothing may depend on a maximised window.
- **Finish the details:** a page title in the browser tab, a favicon, the active link highlighted in the navbar, and a spinner or a disabled button while something is saving.

Before you submit, confirm it yourselves. Open every page of the project one by one, on a laptop and on a phone, signed in as each role, and once as a brand-new user who has no data at all. Whatever looks embarrassing there is exactly what we will see during the demo.

Two hours of design beats two more features. A smaller system that looks finished always scores higher than a bigger system that looks unfinished.

Deliverables at the end of the week

- The **GitHub repository link**, with the migrations and the seeding code inside it, and commits from all three members.
- The **ERD** of your database **as an image**.
- A **ten-minute live demo**: you open the app and walk through one complete story — a real user signs up and does the main job of the system. No slides in those ten minutes.
- A short presentation of about five slides: the idea, the ERD, the features you finished, the AI feature, and the hardest problem you solved.
- **Every member speaks** about code they wrote and answers questions about it.

The AI feature — every project must have one

Every idea in this document ends with AI ideas that fit it, and **you must build one of them** — this is a required part of the project, not an extra, and it is what makes a project stand out in the demo. You do not need to know anything about machine learning: you call a model over HTTP from a normal C# service class, exactly like calling any other API.

Choose one of these three levels

Level	What it is
1 · Simple chat	A chat page inside your app: the user types, the model answers, and the conversation is kept in the session. It knows nothing about your data.
2 · Chat over your own data recommended	The same chat, except that before calling the model you read the relevant rows from your database and put them into the prompt: <i>"Here are our 40 products ... answer the customer using only this list."</i> Now it answers real questions about your system.
3 · A simple agent	You describe two or three of your own C# methods to the model as tools — <code>SearchRooms</code> , <code>GetMyOrders</code> , <code>CreateBooking</code> . The model chooses which one to call, you execute it, you send the result back, and the loop repeats until it can answer. Now the chat can actually do things inside your app.

One rule you must not break

- **Never commit your API key.** Keep it in User Secrets or in `appsettings.Development.json` and put that file in `.gitignore` . A key pushed to GitHub is a security mistake and it costs marks.

The idea. One system that runs the daily life of a private school instead of the pile of Excel sheets it uses today: students, teachers, classes, subjects, attendance and grades all in one place. The admin registers a student and puts them in a class; the teacher opens the class, takes attendance in one screen and enters the marks; the student — or the parent — signs in and sees the timetable, the attendance and the report card without calling anyone.

Why it is worth building. It gives you the cleanest relations in this whole document: a student belongs to a class, a class has many subjects, a subject has one teacher, and attendance and grades sit in the middle of all of it.

Admin

Teacher

Student / Parent

Pages you must build — every page in this table is mandatory

Page	What it does
Home (public)	The school's face: a hero image, a short about section, the latest news, contact details and a login button. This is the page that decides whether your project looks professional.
Login & sign-up MANDATORY	A student or a parent signs up and gets an account; the admin then links them to a class. The role decides which dashboard you land on after login.
Dashboard per role	Admin: counts of students, teachers and classes, plus a chart. Teacher: today's classes. Student: the next lesson, the attendance percentage and the latest grades.
Students list	Search by name or ID, filter by class and by year, sorting and paging, with a photo in every row.
Student details	One profile page: personal data, guardian and phone, the class, attendance percentage, the grade history and the notes written about them.
Add / edit student	A validated form — unique national ID, a sensible birth date, a valid email — with a photo upload.
Classes & subjects	Create a class ("Grade 4 – A"), attach its subjects, and assign a teacher to each subject.
Teachers	A list and a profile page: subjects taught, classes, contact details.
Attendance	The teacher picks a class and a date and marks the whole list present or absent in a single screen , then saves once. This is the page we will look at hardest.
Grades	The teacher enters a mark per student per subject per exam; the student sees the finished report card with a total and a pass/fail state.
Timetable	The weekly schedule of a class as a proper table: days across, periods down, subject and teacher in each cell.
Announcements	The admin publishes news; students and teachers see them on the home page and in their dashboard.
Profile	Edit your own data, change your photo, change your password.

Extras you can add — pick two or three, do them properly

- **Parent account** — a separate login linked to one or more children, showing only their attendance, grades and notes.
- **Report card as PDF** — the student downloads or prints a designed report card instead of a screenshot of a table.
- **Absence notification** — when a teacher marks a student absent, an email or an in-app notification goes to the guardian the same day.
- **Behaviour log** — short positive or negative notes on a student that build a picture over the term.
- **Fees & invoices** — what is due, what is paid, and a printable receipt, plus a chart of collection per month.

AI ideas that fit this system

- **Ask about my child.** A chat that reads the signed-in student's rows from your database and answers questions like "How many days was Ali absent this month?" or "Which subject is he weakest in?"
- **Teacher's comment writer.** The teacher clicks a button on a student; the model reads the marks and the attendance and drafts a paragraph of feedback for the report card, which the teacher can edit before saving.
- **Quiz generator.** The teacher types a subject and a topic and gets five multiple-choice questions with answers, ready to copy.
- **Announcement drafter.** The admin writes three bullet points and gets a polished announcement in Arabic or English.
- **Agent version.** Give the model the tools `GetStudentGrades` , `GetAttendance` and `GetTimetable` , and let the parent simply ask anything in one sentence.

The idea. A public library online. Any visitor can browse the catalogue and see what is on the shelf; a member signs in, borrows a book and gets a due date; the librarian issues and receives copies, chases what is late and calculates the fine. The interesting part of this system is that a **book** and a **copy** are two different things — the library owns three copies of the same title, and only a copy can be borrowed.

Why it is worth building. It is the friendliest idea to make beautiful: covers, categories, ratings and a real search page.

Admin / Librarian

Member

Visitor (browse only)

Pages you must build — every page in this table is mandatory

Page	What it does
Home	A big search bar, new arrivals, the most borrowed books, and the categories as tiles.
Catalogue	A card grid of covers with filters by category, author, language and availability, plus sorting and paging.
Book details	Cover, summary, author, category, how many copies are free right now, a borrow button, and the members' reviews.
Login & sign-up MANDATORY	A visitor signs up and becomes a member; the membership has a start date and an expiry.
My shelf	The member's dashboard: what I have borrowed, when each item is due, how many days are left, my history and my unpaid fines.
Borrow	The librarian issues a free copy to a member, sets the due date, and the copy immediately disappears from the available count.
Return	The librarian receives a copy, the system compares the date with the due date and creates a fine automatically when it is late.
Books admin	Full CRUD on books with a cover upload, plus adding and removing individual copies of a title.
Authors & categories	CRUD, with an author page listing everything they wrote.
Members admin	The list of members, each member's current loans, and the ability to block a member who is far too late.
Reservations	When every copy is out, the member joins a waiting list and is first in line when one comes back.
Reports	The overdue list, the most borrowed titles, the categories nobody reads, loans per month as a chart.
Reviews	A member who has actually borrowed a book may rate it and write a review; the average rating shows on the card.

Extras you can add — pick two or three, do them properly

- **Due-date reminder** — an email or a notification two days before a book is due.
- **ISBN lookup** — the librarian types an ISBN and the title, author and cover are filled in automatically from a public books API.
- **E-books** — a PDF upload on a title that members can read or download in the browser.
- **Wishlist** — save books to read later, and get told when one becomes available.
- **QR code per copy** — printed and scanned at the desk to issue or return instantly.
- **Fine payment** — a simulated payment that clears the fine and prints a receipt.
- **Reading history statistics** — how many books I read this year, and in which categories.

AI ideas that fit this system

- **The librarian chat.** "I want a short novel about the sea, in Arabic, easy language" — you pass your catalogue rows into the prompt and the model recommends three real titles from your database, with a reason for each.
- **Recommendations.** On the book page, "readers who liked this also liked" generated from the member's borrowing history.
- **Auto summary.** The admin pastes the back cover and the model writes a clean four-line description and suggests the category and the tags.
- **Review digest.** Turn twenty reviews into one honest paragraph: what people liked and what they complained about.
- **Agent version.** Tools `SearchBooks`, `GetMyLoans` and `ReserveBook` — the member says "reserve me the next copy of that novel" and it happens.

The idea. A medical centre with several doctors and several specialties. A patient opens the site, picks a specialty, reads the doctors' profiles, chooses a free time slot and books — without a phone call. The doctor opens their screen in the morning and sees the day's list, marks each visit as done and writes the diagnosis and the prescription. The receptionist sees everything and handles walk-in patients.

The hard and interesting part. Time slots: from a doctor's working hours you generate the available times of a day and hide the ones already taken, so two patients can never book the same slot.

Admin / Reception

Doctor

Patient

Pages you must build — every page in this table is mandatory

Page	What it does
Home	The centre, its specialties as tiles, the featured doctors, the working hours, the location and a large "book now" button.
Doctors	A card grid filtered by specialty, by day available and by fee, with a photo and a rating on every card.
Doctor profile	Biography, specialty, qualifications, fee, working hours, patient ratings, and the booking button.
Booking	Pick a doctor, then a date, then a free slot from the generated times — booked slots are visibly disabled — then confirm and get a booking number.
My appointments	The patient's upcoming and past visits, with cancel and reschedule inside the allowed window.
Doctor's day	Today's patients in order with their status (waiting, in progress, done, no-show), and one click to open the visit.
Visit record	The doctor writes the complaint, the diagnosis, the prescription and any notes, and can attach a file. It is saved into the patient's history.
Patient history	Every past visit of one patient in a timeline, readable by that patient and by their doctor — and by nobody else.
Reception desk	The whole day for all doctors on one screen, adding a walk-in patient, and changing an appointment's status.
Admin	CRUD on doctors, specialties, working hours and fees, plus managing user accounts and roles.
Reports	Visits per doctor per week, the busiest hours, the no-show rate, income per specialty as a chart.
Login & sign-up MANDATORY	Sign-up collects the patient's basic medical data: birth date, blood type, chronic conditions, allergies.

Extras you can add — pick two or three, do them properly

- **Confirmation message** — an email on booking and a reminder the day before.
- **Rate the doctor** — a rating form that only opens after a completed visit.
- **Printable prescription** — a designed A5 print of the prescription with the clinic's header.
- **Lab results** — the receptionist uploads a PDF result onto a visit and the patient downloads it.
- **Doctor leave** — a doctor blocks a day, and its slots disappear from booking automatically.

AI ideas that fit this system

- **Which specialty do I need?** The patient describes the problem in plain words and the assistant suggests the specialty and two doctors from your database — with a clear, visible disclaimer that this is not a diagnosis.
- **History summary for the doctor.** One button that turns eight past visits into a five-line briefing before the patient walks in.
- **Booking from a sentence.** "I want Dr. Sara next Tuesday morning" — the model reads the free slots and prepares the booking for the patient to confirm.
- **Prescription tidy-up.** The doctor types quickly and messily; the model formats it into a clean numbered list of medicine, dose and duration for printing.
- **Agent version.** Tools `FindDoctors`, `GetFreeSlots` and `BookAppointment`, wrapped in a chat bubble that lives on every page.

Required here: every AI answer must visibly say it is not medical advice.

The idea. A real online shop for one category of products — electronics, clothes, books, perfumes, whatever your team likes. The visitor browses and filters, adds to a cart, checks out, and then follows the order until it arrives. The admin manages the catalogue, the stock and the orders from a proper dashboard.

Why it is worth building. It is the idea every employer recognises immediately, and it forces you to handle the two things every real system needs: a **cart that survives** across pages, and **stock that must never go negative** when two people buy the last piece.

Visitor

Customer

Admin

Pages you must build — every page in this table is mandatory

Page	What it does
Home	A hero banner, the categories, the featured products, the current offers, and the best sellers. This page must look like a shop, not like a school project.
Products	A card grid with filters by category, price range, brand and rating, with sorting (newest, price, most popular), a search box and paging.
Product details	An image gallery, the description, the price and any discount, the stock state, a quantity picker, add to cart, related products and the reviews.
Cart	Change quantities, remove a line, see the subtotal, shipping and total, apply a coupon — and it survives a page refresh.
Checkout	Choose or add an address, choose shipping, choose payment (cash on delivery or a simulated card), confirm and see the order summary before paying.
Order confirmed	A thank-you page with the order number and the expected delivery date.
My orders	The customer's order list and an order details page with a status timeline: pending, confirmed, shipped, delivered.
Login & sign-up MANDATORY	Sign-up, login, profile, saved addresses and change password.
Admin dashboard	Sales today and this month, the number of orders per status, low-stock alerts, and a sales chart.
Products admin	Full CRUD with multiple image uploads, stock, discounts, and activating or hiding a product.
Categories & coupons	CRUD on categories and brands, plus discount codes with a percentage and an expiry date.
Orders admin	Every order with a status filter, the details of each one, changing the status, and cancelling with a reason.
Reviews	Only a customer who bought the product may review it; the admin can hide an abusive review.

Extras you can add — pick two or three, do them properly

- **Wishlist** — a heart on every card, and a page of everything saved.
- **Live search suggestions** — a dropdown of matching products as you type.
- **Invoice PDF** — a designed invoice the customer downloads after delivery.
- **Order emails** — a message on confirmation and on every status change.
- **Product comparison** — pick two or three products and see their specs side by side.
- **Multi-vendor** — sellers register, add their own products and see only their own orders.
- **Dark mode** — a real one, remembered per user.

AI ideas that fit this system

- **Shopping assistant.** "I need a gift for a ten-year-old, under 500 pounds" — the model receives your product rows and recommends three real items with a link to each.
- **Description writer.** The admin types a product name and three features and gets a selling description plus SEO keywords, editable before saving.
- **What do people say?** A one-paragraph summary of all the reviews of a product, with the good and the bad separated.
- **Smart search.** The customer types a whole sentence instead of a keyword and you translate it into filters — category, price range, brand — before querying.
- **Agent version.** Tools `SearchProducts`, `AddToCart` and `TrackOrder`, so the chat can fill the cart and answer "where is my order?" without leaving the page.

The idea. The website of a hotel. The guest enters a check-in date, a check-out date and the number of guests, and sees only the rooms that are **actually free for those nights**, books one, adds breakfast or an airport pickup, and gets a voucher. Reception handles arrivals and departures; the admin manages the rooms, the photos and the prices.

The hard and interesting part. Availability over a date range: a room is free only if no existing booking overlaps the requested nights. Get that query right and you have learned something most juniors cannot do.

Guest

Receptionist

Admin

Pages you must build — every page in this table is mandatory

Page	What it does
Home	A full-width hotel photo, a short story about the place, the facilities as icons, a gallery, and the availability search box sitting on top of the hero.
Search results	The rooms free for the chosen dates as cards, filtered by type, capacity, price and view, each showing the total price for the whole stay — not just the nightly rate.
Room details	A photo gallery, the size, the bed type, the amenities, the price per night, the cancellation policy, and a booking button.
Booking	Confirm the dates and guests, add the extras, see the calculated total night by night, and confirm to get a booking reference.
My bookings	Upcoming and past stays, cancellation inside the policy window, and a printable voucher.
Reception	Today's arrivals and departures with a check-in and a check-out button, plus creating a walk-in booking on the spot.
Rooms admin	CRUD on rooms and room types with multiple image uploads, amenities, capacity, and taking a room out of service for maintenance.
Pricing	A base price per room type and a seasonal price for a date range — high season costs more, and the booking total must respect it.
Bookings admin	All the bookings with filters by status and by date, the details of each, and cancelling with a reason.
Dashboard	Occupancy rate, revenue this month, arrivals today, and a chart of bookings per month.
Reviews	A guest who has actually checked out may rate the stay; the average appears on the home page.
Login & sign-up MANDATORY	Sign-up, login and a profile with passport or ID number and nationality.

Extras you can add — pick two or three, do them properly

- **Occupancy calendar** — a month grid showing which rooms are taken on which nights.
- **Discount codes** — a promo code applied at booking with a percentage and an expiry.
- **Housekeeping board** — clean, dirty or being cleaned, updated by the staff.
- **Voucher by email** — a designed confirmation sent to the guest's inbox.
- **Two languages** — Arabic and English with a real switch, using resource files.
- **Map and directions** — an embedded map with the hotel's exact location.
- **Loyalty points** — points on each stay, redeemable against the next one.

AI ideas that fit this system

- **Concierge chat.** Answers questions about the hotel — check-in time, is breakfast included, is there parking — from a facts sheet you write and put into the prompt.
- **Book by sentence.** "Two nights next weekend, sea view, under 2000 a night" turns into a date range and filters, and the assistant shows the matching free rooms.
- **Room descriptions.** The admin adds the facts and the model writes the tempting paragraph that sells the room.
- **Review summary.** Turn all the reviews into "guests loved the breakfast and complained about the lift" for the dashboard.
- **Agent version.** Tools `CheckAvailability`, `GetRoomPrice` and `CreateBooking` — the guest books an entire stay from the chat bubble.

The idea. A restaurant with two things happening at once: people ordering food for delivery or pickup, and people reserving a table for tonight. The customer browses a menu with photos, builds an order with options — size, extras, no onions — and follows it while it is being prepared. The staff work on a live screen that shows the new orders and moves them through the kitchen.

Why it is worth building. It is the most visual idea in the list, and the order options give you a genuinely interesting price calculation instead of a plain quantity times price.

Customer

Staff / Cashier

Admin

Pages you must build — every page in this table is mandatory

Page	What it does
Home	A hero photo of the food, today's offers, the top dishes, the opening hours, and two clear buttons: order now, or reserve a table.
Menu	Dishes grouped by category — starters, mains, drinks, desserts — as photo cards, with a category filter and a search box.
Dish details	The photo, the ingredients, the size options and the paid extras, all of which change the displayed price live before adding to the cart.
Cart & checkout	Edit quantities, add a note for the kitchen, choose delivery with an address or pickup, choose a payment method, and confirm.
Order tracking	A visual timeline: received, being prepared, on the way, delivered — with the time of each step.
Table reservation	Pick a date, a time and the number of guests, see which tables are free at that hour, choose one and confirm.
My orders & reservations	The customer's history, with reordering in one click and cancelling a reservation.
Staff screen	The live list of orders ordered by time with a big status button on each, plus today's reservations. Auto-refresh is a nice touch here.
Menu admin	CRUD on categories, dishes with a photo, prices, options and extras, and marking a dish as sold out.
Tables admin	CRUD on tables — number, capacity, indoor or outdoor — and the opening hours that reservations respect.
Offers	A discounted dish or a combo, shown as a badge on the card and applied in the cart.
Reports	Sales today, the best-selling dishes, the busiest hours, and a chart of orders per day.
Login & sign-up MANDATORY	Sign-up, login, profile and saved addresses.

Extras you can add — pick two or three, do them properly

- **QR menu per table** — scanning it opens the menu with the table number already attached to the order.
- **Kitchen ticket** — a printable slip for each order with only what the kitchen needs.
- **Loyalty points** — points on every order, redeemable for a free dish.
- **Rate the dish** — a rating after delivery, shown as stars on the menu card.
- **Branches** — several branches, each with its own menu and its own orders.
- **Delivery zones** — a different delivery fee and time per area.
- **Live status without refresh** — update the tracking page with SignalR or a timer.

AI ideas that fit this system

- **The waiter chat.** "Something spicy, no chicken, under 200 pounds, for two people" — the model reads your menu rows and suggests a full meal.
- **Order from one sentence.** "Two large margheritas, one without olives, and a Pepsi" becomes a filled cart the customer only has to confirm.
- **Appetising descriptions.** The admin enters the ingredients and the model writes the line that makes people order it.
- **Smart upsell.** On the cart page, a suggestion of the drink or dessert that fits what is already in it — with a reason, not a random product.
- **Agent version.** Tools `SearchDishes`, `AddToOrder` and `ReserveTable`, so one chat can both feed the customer and seat them.

The idea. A recruitment site with two completely different kinds of user. A company registers, publishes a vacancy and reviews the people who applied; a job seeker builds a profile, uploads a CV, searches with real filters and follows the status of every application. The admin approves companies so the site does not fill with fake vacancies.

Why it is worth building. Two profile types on top of one Identity user is exactly the problem real systems have, and the application status flow gives you a clean, demo-friendly story from both sides.

Job seeker

Company / HR

Admin

Pages you must build — every page in this table is mandatory

Page	What it does
Home	A search bar for a job title and a city, the categories as tiles with a count on each, the newest vacancies, and the featured companies.
Jobs list	Filters by category, city, type (full time, part time, remote), experience level and salary range, with sorting and paging.
Job details	The description, the requirements, the skills, the salary range, the company card, how many people applied, and an apply button that changes to "applied" afterwards.
Login & sign-up MANDATORY	One sign-up page, two paths: I am looking for a job, or I am hiring. Each one leads to a different profile and a different role.
Seeker profile	Personal data, a photo, a summary, skills, work experience, education, and a CV upload as PDF .
My applications	Everything I applied to with its status: applied, under review, interview, accepted, rejected — with the date of each change.
Company dashboard	My open vacancies, the number of applicants on each, new applications this week, and a chart.
Post a job	A validated form for a vacancy with a closing date; after that date it disappears from the list automatically.
Applicants	Per vacancy: the list of applicants, opening a CV in the browser, changing a status, writing a private note, and filtering by status.
Company page	A public profile — logo, about, industry, size, location — and every vacancy they currently have open.
Companies list	All the companies with a search box and a filter by industry.
Admin	Approving or rejecting companies and vacancies, CRUD on categories, cities and skills, managing users, and site-wide reports.

Extras you can add — pick two or three, do them properly

- **Saved jobs and alerts** — save a search and get told when a matching vacancy appears.
- **Status emails** — the seeker is told the moment a company moves them forward.
- **Interview scheduling** — the company proposes a date and the seeker accepts it.
- **A short test** — a few multiple-choice questions attached to a vacancy, scored automatically.
- **Public profile link** — a shareable read-only page of the seeker's CV.
- **Featured vacancy** — a paid-looking highlight that puts a job at the top of the list.
- **Company reviews** — employees rate the workplace out of five.

AI ideas that fit this system

- **CV reader.** The seeker uploads a PDF, you extract its text, and the model fills the skills, the experience and the education into the profile form for the user to review.
- **Matching score.** For every applicant on a vacancy, a percentage and one line of justification — "8 of 10 required skills, 3 years of the 5 asked for".
- **Cover letter writer.** The seeker clicks apply and gets a draft letter written from their profile and that specific vacancy.
- **Vacancy writer.** HR types five bullet points and gets a complete, well-structured job description.
- **Agent version.** Tools `SearchJobs`, `GetMyApplications` and `ApplyToJob` — "find me remote .NET jobs in Cairo and apply to the best three".

The idea. A gym that stops running on paper. A member subscribes to a plan and can see exactly how many days are left, books a place in a class before it fills up, follows the workout and diet plan their trainer wrote, and watches their own weight change over the weeks. The trainer manages their classes and marks who showed up. The admin sees the money and the memberships that are about to expire.

The hard and interesting part. A class has a capacity: bookings must stop at the limit, and a cancellation must free the place for someone else.

Member

Trainer

Admin

Pages you must build — every page in this table is mandatory

Page	What it does
Home	A strong hero image, what the gym offers, the trainers, a photo gallery, the plans, and a "join now" button.
Plans	The membership plans as pricing cards — monthly, quarterly, yearly — with what each one includes and a subscribe button.
Login & sign-up MANDATORY	Sign-up, login and a profile: personal data plus height, weight, goal (lose weight, build muscle, stay fit) and fitness level.
Member dashboard	The active subscription with the days remaining, the next booked class, attendance this month and the current weight.
Class schedule	The week as a table: the class, the trainer, the time, and how many places are left. Booking and cancelling happen here, and a full class is visibly disabled.
My bookings	The classes I booked, upcoming and past, with cancellation up to a few hours before.
Trainers	A list and a profile: specialty, certificates, the classes they run, and a rating.
Trainer dashboard	My classes today, the list of people who booked each one, and marking attendance.
Workout & diet plan	The trainer builds a plan for a member — exercises, sets, repetitions, and meals per day — and the member opens it on their phone at the gym.
My progress	The member records weight and measurements over time and sees a real chart of the change.
Admin	CRUD on plans, classes, trainers and members, plus subscriptions and payments.
Reports	Active members, income this month, memberships expiring this week, and the most popular classes.
Check-in	The member arrives and the entry is recorded — and a member whose subscription has expired is refused with a clear message.

Extras you can add — pick two or three, do them properly

- **QR check-in** — every member has a QR code on their phone that reception scans.
- **Freeze a subscription** — pause it for a period and extend the end date by the same amount, and warn the member three days before it ends.
- **Personal training** — one-to-one sessions booked with a specific trainer at a chosen hour.
- **Body measurements** — chest, waist and arms tracked over time next to the weight, with a printable receipt for every payment.

AI ideas that fit this system

- **Plan generator.** The trainer enters the goal, the level and the available days, and the model drafts a full week of workouts that the trainer edits and assigns.
- **The coach chat.** The member asks about exercises, form or nutrition and gets an answer — with a visible line saying this is general information, not medical advice.
- **Which class suits me?** The model reads the member's goal and free days and recommends classes from your real schedule.
- **Progress in words.** A monthly paragraph written from the weight log and the attendance rows: what improved, what slipped, what to focus on next.
- **Agent version.** Tools `GetSchedule` and `BookClass` — "book me yoga on Tuesday evening" simply works.